

Vocabulary Builder Assessment

1. Executive Summary

Vocabulary Builder is a functional early-stage learning product, not merely a prototype. It already supports authentication, dictionary lookup, personal vocabulary storage, favorites, preferred quiz definitions, pronunciation audio, quizzes, and basic quiz history.

Its strongest foundation is the sensible core relationship between canonical words and user-specific vocabulary. The main weakness is that the application still behaves like three adjacent tools—lookup, word list, and quiz—rather than a continuous learning system.

The best direction is an incremental evolution into a personal vocabulary practice application centered on this loop:

Discover → Save → Review → Practice → Measure → Resurface weak words

A rewrite is neither necessary nor advisable. The current .NET/Angular monolith is appropriate for a small team. First stabilize security, contracts, and testing; then introduce a clear learning-state model and daily review workflow.

Confirmed baseline:

- .NET solution builds with zero warnings and zero errors.
- Angular production build succeeds.
- Angular reports a style-budget warning for `word-lookup.component.scss`.
- The Angular unit run did not complete within 120 seconds.
- There is no backend test project.
- No code was modified.

2. Current Product Assessment

Current purpose

The application lets an authenticated user:

- Look up English words through `dictionaryapi.dev`.
- Cache dictionary responses locally.

- Add words to a personal vocabulary.
- Browse and search saved words.
- Favorite words.
- Select a preferred definition for quizzes.
- Hear pronunciation audio or browser speech synthesis.
- Take multiple-choice word/definition quizzes.
- View recent quiz scores.

The implemented flow is visible across

[WordService.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Services/WordService.cs),
[word-lookup.component.ts](F:/Source/VocabularyApp/VocabularyApp.UI/src/app/components/
word-lookup/word-lookup.component.ts), and
[QuizService.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Services/QuizService.cs).

Strongest existing features

- Canonical words are separated from user-owned vocabulary.
- Multiple definitions and parts of speech are supported.
- External results are cached for later users.
- Preferred definitions reduce quiz ambiguity.
- Favorites are implemented end-to-end.
- Quiz correctness is determined server-side.
- User ownership is checked on protected vocabulary operations.
- Vocabulary lookup provides autocomplete and definition/example search.
- Audio has a practical speech-synthesis fallback.

Incomplete or confusing areas

Confirmed:

- “Save for Later” is visible but has no click handler in [word-lookup.component.html (line 147)](F:/Source/VocabularyApp/VocabularyApp.UI/src/app/components/word-lookup/word-lookup.component.html:147).
- The dashboard advertises analytics, preferences, and administration as inactive concepts rather than providing a focused learning home in [dashboard.component.ts (line 17)](F:/Source/VocabularyApp/VocabularyApp.UI/src/app/components/dashboard/dashboard.component.ts:17).
- `PersonalNotes`, `SampleSentence`, and `ChatHistory` exist in the model but are not usable through the current UI/API.
- Quiz attempts are recorded, but `UserWord.CorrectAnswers`, `TotalAttempts`, `LastReviewedAt`, and `LastCorrectAt` are not updated during submission. Displayed per-word accuracy therefore remains stale.
- Users cannot delete or archive a saved word.

- There is no review queue, due state, mastery status, or next recommended action.
- Registration creates a token in the backend response, but the UI redirects to login rather than establishing the returned session.
- Documentation is outdated: it calls the Angular UI and quiz service future work even though both now exist.

Realistic product direction

A strong, realistic product is a **personal vocabulary acquisition and retention coach**:

- Capture unfamiliar words quickly.
- Choose the meaning the user wants to learn.
- Add a note or personal example.
- Practice a short daily queue.
- Emphasize weak or overdue words.
- Show simple, actionable progress.

It should not initially become a general dictionary, classroom LMS, social network, or open-ended AI platform.

3. Architecture Strengths

The architecture is a conventional modular monolith:

```
Angular standalone UI
    ↓ HTTP + JWT
ASP.NET Core controllers
    ↓
Application services
    ↓
EF Core DbContext
    ↓
SQL Server
```

WordService → external dictionary API

Strengths include:

- The UI, API, and data model are separate projects.
- Controllers depend on service interfaces.
- EF entities are not returned directly to the frontend.
- DTOs exist for words, vocabulary, users, and quizzes.
- Authentication and authorization use standard ASP.NET Core middleware.
- The external dictionary is accessed through `HttpClient`.

- SQL relationships and useful uniqueness constraints are configured centrally in [ApplicationDbContext.cs](F:/Source/VocabularyApp/VocabularyApp.Data/ApplicationDbContext.cs).
- The quiz never sends its correct option identifier to the client.
- The current deployment remains simple enough for one small team.

These are good foundations. More layers or a microservice split would add cost without solving the present problems.

4. Architecture Weaknesses

Oversized responsibilities

[WordService.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Services/WordService.cs) currently handles:

- External provider access.
- Canonical dictionary caching.
- Entity creation.
- Definition and part-of-speech mapping.
- Vocabulary commands.
- Favorite and preferred-definition updates.
- Vocabulary querying and search.
- DTO projection.

[WordLookupComponent](F:/Source/VocabularyApp/VocabularyApp.UI/src/app/components/word-lookup/word-lookup.component.ts) contains 780 lines of state and behavior for:

- Lookup.
- Autocomplete.
- Dictionary result mapping.
- Vocabulary browsing and filtering.
- Definition selection.
- Favorites.
- Audio.
- Toast presentation.
- HTML highlighting.

Its 443-line template repeats definition rendering and includes a modal, toast host, search screen, dictionary detail, and collection screen.

Weak boundaries

- Interfaces frequently return `ServiceResult<object>`, losing compile-time contract safety.
- Controllers repeatedly extract the user ID, construct anonymous envelopes, catch all exceptions, and map failures manually.
- There are multiple conflicting response types:
 - `ApiResponse`
 - **generic and non-generic** `ApiResult`
 - `ServiceResult<T>`
 - **anonymous** `{ success, data, error }` objects
- `ApiResult.ErrorResult(object)` **throws** `NotImplementedException`.
- `ApiResult<T>` and `ChangePasswordRequest` are declared inside `UsersController.cs`, despite similar DTO files already existing.
- `DTOs/ApiResult.cs` declares its type in the `Models` namespace, which is misleading.
- Business failures do not distinguish validation, conflict, not-found, or infrastructure errors.

Growth hazards

- Static in-process quiz sessions disappear on restart and do not work reliably with multiple API instances.
- External lookup and persistence are not atomic. Concurrent lookup of the same new word can violate the unique word index after the external call.
- External requests have no explicitly visible provider abstraction, retry policy, timeout policy, or cancellation tokens.
- Search repeatedly calls `ToLower()` in SQL and loads definitions through broad includes.
- The UI fetches up to 1,000 vocabulary entries and performs browsing/search locally, bypassing the backend pagination already present.
- Routes eagerly import every component.
- Authentication headers are built manually in every `ApiService` request instead of using an interceptor.
- Components subscribe directly without a consistent lifecycle/state pattern.

Recommended boundaries

Keep one deployable backend, but split responsibilities into:

- `DictionaryLookupService`
- `DictionaryProvider` **or** `IDictionaryClient`
- `VocabularyService`
- `QuizGenerationService`
- `QuizAttemptService`
- **Later,** `ReviewSchedulingService`

On the frontend, introduce feature services and focused components:

- VocabularyApi
- QuizApi
- WordSearchComponent
- WordDetailComponent
- VocabularyListComponent
- VocabularyCardComponent
- PreferredDefinitionDialog
- Shared application header and toast host

5. Extensibility Assessment

Capability	Current readiness	Needed change	Timing
Favorites/priority	High	Add favorite filtering and optionally a separate priority field	Soon
Personal notes	Medium	Field exists; add typed update endpoint and editor	Soon
Example sentences	Medium	Entity exists; add CRUD and decide user-created versus provider examples	Soon
Weak-word tracking	Medium	Derive initially from attempts; later persist scheduling state	Soon
Mastery tracking	Low–Medium	Define mastery semantics; add learning state beyond aggregate counters	Soon
Daily review queues	Low	Add due dates, scheduling policy, and review endpoint	Soon after learning model
Spaced repetition	Low	Add per-user-word scheduling fields or a separate review-state entity	Phase 4
Word collections	Low	Introduce collection and membership entities; avoid overloading favorites	Later
Advanced quizzes	Medium	Quiz infrastructure exists; isolate generators and support typed answers	Later
Progress analytics	Medium	Quiz history exists, but counters and session metadata must be reliable first	Soon/Later
AI vocabulary coach	Low	Define safe use cases, provider interface, cost controls, and context boundaries	Much Later

Conversational practice	Low	Chat schema is only a placeholder; needs sessions, goals, evaluations, and moderation	Much Later
-------------------------	-----	---	------------

Recommended learning-state model

Do not add a large SRS framework yet. Start with either fields on `UserWord` or a one-to-one `ReviewState`:

- `MasteryLevel`
- `NextReviewAt`
- `ReviewIntervalDays`
- `ConsecutiveCorrect`
- `LapseCount`
- `LastReviewedAt`

Retain immutable `QuizResult` records as event history. Update the review state transactionally when an answer is submitted.

6. Data Model Issues

Good decisions

- `Word` is canonical and reusable across users.
- `WordDefinition` supports multiple senses.
- `UserWord` owns user-specific learning state.
- Quiz results are stored per question, enabling later analytics.
- Preferred definition is a useful explicit user choice.

Confirmed problems

Contradictory `UserWord` design

[`UserWord.cs`](F:/Source/VocabularyApp/VocabularyApp.Data/Models/UserWord.cs) contains:

- Persisted `PartOfSpeechId`, despite comments claiming part of speech was removed.
- [NotMapped] `CreatedAt`, duplicating persisted `AddedAt`.
- [NotMapped] `CustomDefinition`.
- [NotMapped] `DifficultyLevel`.
- Persisted `PersonalNotes` without supporting endpoints.
- Learning counters that quiz submission does not update.

The DTOs still expose removed or non-persisted fields. This creates false API capabilities and makes migrations harder to reason about.

Incorrect entity concerns

[WordDefinition.cs (line 27)](F:/Source/VocabularyApp/VocabularyApp.Data/Models/WordDefinition.cs:27) has [NotMapped] Success and Data. Response-envelope fields do not belong on an EF entity.

Part-of-speech fallback

Unknown provider parts of speech silently become Noun. The frontend recognizes determiner and exclamation, but the seeded database does not. This can corrupt semantic categorization rather than merely losing information.

Redundant ownership keys

SampleSentence stores both UserId and UserWordId, while UserWord already determines ownership. This permits inconsistent rows unless explicitly validated. Prefer ownership through UserWord, unless direct user querying has a measured need.

Quiz semantics

- QuizType supports five modes, but every persisted attempt is Definition.
- ResponseTimeSeconds is always zero.
- QuizSessionId groups attempts but there is no QuizSession entity for mode, start time, completion, or abandonment.
- ChatHistory.Context and Role are unconstrained strings and are premature for an unimplemented feature.

Practical improvements

Now:

- Remove obsolete [NotMapped] placeholder fields or implement them deliberately.
- Remove response properties from WordDefinition.
- Normalize word text and user identifiers before storage.
- Validate preferred definition consistency.
- Add database check constraints for nonnegative counts.
- Decide whether one user may save multiple senses of the same word.

Later:

- Add QuizSession.
- Add explicit review state.
- Add collection tables only when collections are being built.
- Replace raw chat rows only when the AI use case is defined.

7. Backend/API Issues

Critical inconsistencies

- `POST /api/words/add` is described as an admin endpoint but has neither `[Authorize]` nor an admin policy in `[WordsController.cs (line 58)](F:/Source/VocabularyApp/VocabularyApp.WebApi/Controllers/WordsController.cs:58)`.
- That endpoint ignores a failed service result and always returns HTTP 200.
- Lookup converts every service failure—including external provider failure—into HTTP 404.
- Most domain failures return HTTP 400 even when they are conflicts or missing resources.
- The frontend expects both `message` and `error`, reflecting inconsistent backend contracts.
- `CreateUserAsync` returns `ex.Message` to the controller on registration failure, potentially exposing database details.
- No centralized exception handler or problem-details response exists.
- No rate limiting exists for login, registration, dictionary lookup, or writes.
- `StartQuizRequestDto`, quiz answers, and some word requests have little or no declarative validation.
- No cancellation tokens are propagated.

Authentication observations

Good:

- Protected quiz and vocabulary endpoints use authorization.
- JWT validation checks issuer, audience, signature, lifetime, and uses zero clock skew.
- Ownership predicates include both user and resource ID.

Weak:

- User ID claim extraction is duplicated in nearly every action.
- `ValidateTokenAsync` in `IUserService` appears unnecessary because middleware already validates tokens.
- There is no refresh/revocation strategy. This is acceptable initially if token lifetime remains short, but should be an explicit policy.
- Registration/login enumeration behavior differs: registration reveals whether username/email exists; this may be acceptable product behavior but should be deliberate.

Recommended API shape

Use one typed envelope—or standard `ProblemDetails` errors plus typed successful bodies.

Example resources:

- `POST /api/v1/vocabulary`
- `GET /api/v1/vocabulary`
- `PATCH /api/v1/vocabulary/{id}`
- `DELETE /api/v1/vocabulary/{id}`
- `POST /api/v1/quiz-sessions`
- `POST /api/v1/quiz-sessions/{id}/submission`
- `GET /api/v1/reviews/today`

Add a `CurrentUserId` helper or base controller, centralized exception mapping, and typed result codes. Avoid introducing MediatR or a repository layer merely for convention.

8. Frontend/UI Architecture Issues

Primary concern

`WordLookupComponent` is effectively an entire feature application. It should be decomposed before notes, collections, and review controls are added.

Suggested structure:

```
VocabularyPage
├── VocabularyHeader
├── WordSearch
│   ├── SearchSuggestions
│   └── WordDetail
│       └── DefinitionGroup
├── VocabularyBrowser
│   ├── VocabularyFilters
│   └── VocabularyCard
└── PreferredDefinitionDialog
```

Other confirmed issues

- Extensive `any` usage bypasses the otherwise useful TypeScript models.
- HTTP error extraction is duplicated throughout components.
- Autocomplete has no debounce or cancellation; older HTTP responses can overwrite newer input.
- Vocabulary loads 1,000 records client-side.
- Search suggestions are visually structured but lack proper combobox/listbox keyboard semantics.
- The dashboard uses clickable `<div>` elements, which are not keyboard-accessible.

- Toast presentation exists only inside the vocabulary page, so it is not truly application-wide.
- Auth tokens are stored in `localStorage`, increasing the impact of any XSS vulnerability.
- The guard navigates imperatively instead of returning a `UrlTree`.
- Routes are not lazy-loaded.
- Dashboard subscriptions are not explicitly lifecycle-managed.
- Login imports the password component, while signup duplicates its own password UI.
- API endpoints are hand-built strings in page components.

Use feature-local state services or signals before adopting a global state library. NgRx would be unnecessary at this size.

9. UI/UX Issues

The intended journey exists only partially:

Step	Current experience
Search	Strong: lookup, autocomplete, grouped definitions, audio
Save	Functional but ambiguous across “Add,” “Save for Later,” favorites, and preferred definition
Learn	Weak: no notes workflow, personal examples, goals, or explanation of what “learned” means
Review	Quiz is manually entered, not scheduled or personalized
Improve	Recent scores exist, but no weak-word feedback or next action

Specific usability concerns

- The dashboard’s only active destination is “Vocabulary Builder”; the quiz is hidden one level deeper.
- “Save for Later” appears actionable but does nothing.
- “Pick Quiz Definition” exposes an implementation concept rather than a learner-centered phrase such as “Choose the meaning I’m learning.”
- Favoriting and saving have unclear semantic differences.
- No persistent application navigation connects Home, My Words, Review, and Progress.
- After saving a word, there is no suggested next step such as adding a note or reviewing it.
- Quiz completion shows correctness but does not explain which words will be reviewed again.
- Empty states do not establish a learning goal.
- Garbled emoji visible in source indicate an encoding/documentation consistency problem and may render incorrectly depending on the deployed pipeline.

- No clear loading/error state is shown when the vocabulary list fails; it becomes an empty list.

Recommended cohesive navigation

- **Today** — due review count and one primary action.
 - **Discover** — search and save.
 - **My Words** — filter, favorite, annotate, archive.
 - **Review** — daily queue and optional custom quiz.
 - **Progress** — short actionable trends.
-

10. Testing and Reliability

Current state

- There is no backend test project.
- Most Angular specs only test component/service creation.
- [word-lookup.component.spec.ts](F:/Source/VocabularyApp/VocabularyApp.UI/src/app/components/word-lookup/word-lookup.component.spec.ts) has some meaningful mapping, filtering, highlighting, and state tests.
- No quiz component spec exists.
- No password-input spec exists.
- No API integration, database integration, end-to-end, or external-provider contract tests exist.
- The Angular test command timed out after 120 seconds without producing a final result.
- Both production builds succeed; Angular has one SCSS budget warning.

Highest-value missing tests

1. User registration, login, password change, and protected endpoint integration tests.
2. User A cannot read or mutate User B's vocabulary or quiz session.
3. Dictionary cache miss, cache hit, provider failure, unknown part of speech, and concurrent lookup.
4. Add vocabulary duplicate/conflict behavior.
5. Preferred definition must belong to the same word.
6. Quiz generation does not expose correct answers.
7. Submission rejects another user's session and invalid option IDs.
8. Submission persists attempts and updates review counters atomically.
9. Vocabulary paging, search, favorites, and deletion.
10. One end-to-end journey: register → search → save → review → see result.

Small-team strategy

- Favor backend integration tests using a real relational test database or disposable SQL Server container over extensive service mocking.
 - Unit-test pure scheduling, scoring, normalization, and mapping rules.
 - Keep a small Angular component/service suite for interaction logic.
 - Add 3–5 Playwright smoke journeys.
 - Run builds and fast tests on every pull request.
 - Add coverage reporting later; initially measure whether critical behaviors are protected, not a blanket percentage.
-

11. Security / Production Readiness

Critical findings

- A JWT signing secret is committed in [appsettings.json (line 6)](F:/Source/VocabularyApp/VocabularyApp.WebApi/appsettings.json:6). Treat it as compromised, rotate it, and load production secrets from environment/configuration providers.
- Passwords use salted SHA-256 in [PasswordHelper.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Helpers/PasswordHelper.cs), which is fast and unsuitable for password storage. Use ASP.NET Core `PasswordHasher<TUser>`, `Argon2id`, `bcrypt`, or `PBKDF2` with an upgrade path.
- The canonical dictionary write endpoint is publicly accessible.
- HTTPS redirection is commented out in [Program.cs](F:/Source/VocabularyApp/VocabularyApp.WebApi/Program.cs).
- Production CORS includes a plain HTTP hosting origin.
- Swagger is enabled unconditionally.

High-priority production gaps

- No rate limiting or login lockout.
- No security-header policy.
- No health/readiness endpoints.
- No startup migration strategy or deployment documentation.
- No structured correlation/request identifiers.
- No redaction policy for user content or externally supplied values.
- No resilience policy around the dictionary provider.
- Static quiz sessions fail across restarts and horizontally scaled instances.
- JWT in `localStorage` is vulnerable to token theft through XSS. An `HttpOnly` secure-cookie strategy is safer if frontend and API deployment topology supports it.
- No backup, retention, or restore assumptions are documented.

- The root directory contains an untracked-looking credential-named file, `SmarterASP.NET Password.txt`. It is not in `git ls-files`, which is good, but credentials should not be stored in the repository directory at all.

12. Recommended Feature Enhancements

Feature	User benefit	Difficulty	Dependencies	Timing
Daily review queue	Gives users an obvious next action and builds a habit	Medium	Review state and reliable attempt updates	Soon
Notes and personal examples	Makes vocabulary personally meaningful and memorable	Low–Medium	Typed vocabulary update API	Soon
Weak-word review	Focuses practice where it matters most	Medium	Correct attempt aggregation	Soon
Archive/remove word	Lets users manage mistakes and stale items	Low	Delete/archive API and ownership tests	Soon
Favorites filter	Makes existing favorites useful	Low	Query/filter support	Soon
Basic mastery levels	Shows learning progress per word	Medium	Defined mastery policy	Soon
Simple progress screen	Reinforces consistency and shows due/weak words	Medium	Reliable quiz and review data	Later
Review modes	Recognition, recall, and usage practice	Medium	Pluggable question generators	Later
Collections/tags	Supports exam, book, topic, or course vocabulary	Medium	Collection entities and filtering	Later
Daily goal/streak	Encourages consistency	Medium	Review completion events and timezone decision	Later
Import/export	Helps serious users bring or retain vocabulary	Medium	Validation and duplicate policy	Later
AI explanations	Rephrases difficult meanings and generates examples	Medium–High	Provider abstraction, cost/safety controls	Much Later

Conversational practice	Exercises active vocabulary in context	High	AI platform, session model, evaluation policy	Much Later
----------------------------	---	------	---	---------------

13. Technical Debt Priorities

Critical

- **Replace SHA-256 password hashing.** Leaving it allows efficient offline password cracking after a database leak.
- **Rotate and externalize committed JWT secrets.** Leaving them permits token forgery wherever the committed key is deployed.
- **Protect or remove the public canonical write endpoint.** Leaving it permits dictionary pollution and database abuse.
- **Make quiz submission update learning state transactionally.** Leaving it means accuracy and mastery features are built on incorrect data.

High

- **Unify API response and error contracts.** Otherwise every new endpoint increases frontend branching and ambiguity.
- **Add backend integration tests.** Otherwise security and schema refactors have no dependable safety net.
- **Split dictionary and vocabulary responsibilities in `wordService`.** Otherwise review, notes, and collection work will compound coupling.
- **Decompose `wordLookupComponent`.** Otherwise small UI changes continue to affect unrelated states and templates.
- **Replace static quiz-session storage.** Otherwise deployments and multiple instances invalidate active quizzes.
- **Remove or implement placeholder entity/DTO fields.** Otherwise developers cannot tell which product capabilities are real.
- **Handle concurrent dictionary lookup and persistence.** Otherwise ordinary concurrent traffic can produce constraint failures.
- **Introduce centralized exception handling and validation.** Otherwise status codes and information exposure remain inconsistent.

Medium

- Use typed service results rather than `object`.
- Add API client feature services and an authentication interceptor.
- Restore true server-side pagination and search.

- Debounce/cancel autocomplete.
- Add word deletion/archive.
- Normalize case consistently in storage and queries.
- Add provider timeouts/resilience.
- Replace duplicated definition template markup.
- Add accessible navigation and autocomplete semantics.
- Update README and HTTP examples.
- Resolve Angular test-runner performance and the SCSS budget warning.

Low

- Remove stale comments and commented-out UI.
- Replace console logging with environment-aware diagnostics.
- Consolidate naming conventions and namespaces.
- Remove stray package-lock files outside the Angular project if unused.
- Replace encoding-damaged emoji with verified UTF-8 or stable icons.

14. Phased Roadmap

Phase 1 — Stabilize and clean up

Goals: Close security gaps and make existing behavior trustworthy.

Tasks:

- Rotate/externalize secrets.
- Migrate password hashes safely.
- Protect/remove canonical word writes.
- Centralize errors and validation.
- Fix quiz counter/state updates.
- Remove dead DTO/entity fields.
- Add delete/archive vocabulary behavior.
- Update documentation.

Dependencies: Decide current production deployment and whether real users already have SHA-256 hashes.

Risks: Password migration and schema cleanup require backward compatibility.

Benefit: Safe base for all future features.

Do not do yet: AI, collections, advanced analytics, microservices.

Phase 2 — Improve architecture and testing

Goals: Establish safe seams for continued development.

Tasks:

- Add backend integration-test project.
- Extract dictionary provider and vocabulary service.
- Type service responses.
- Add current-user accessor and global exception handling.
- Decompose the word-lookup page.
- Introduce feature API clients/interceptor.
- Add Playwright smoke journeys.
- Persist quiz-session state or redesign submission using signed/persisted session data.

Dependencies: Stable contracts from Phase 1.

Risks: Refactoring can subtly change envelopes or frontend behavior.

Benefit: Lower regression risk and faster feature delivery.

Do not do yet: Generic repository/unit-of-work layers, CQRS framework, global frontend store.

Phase 3 — Improve UI/UX and learning workflow

Goals: Turn the separate features into one cohesive product.

Tasks:

- Replace the dashboard with a “Today” home.
- Add persistent navigation.
- Clarify Save, Favorite, and Learn semantics.
- Remove or implement “Save for Later.”
- Add notes and personal examples.
- Add archive/delete and useful empty states.
- Improve accessibility and responsive interaction.
- Use backend filtering/paging.

Dependencies: Vocabulary API boundary and component decomposition.

Risks: Navigation changes can disorient current users.

Benefit: Clear discover-to-review journey.

Do not do yet: Visual analytics dashboards without reliable learning data.

Phase 4 — Add high-value learning features

Goals: Improve retention rather than merely storing words.

Tasks:

- Add review-state model.
- Implement a simple spaced-repetition policy.
- Add daily due queue.
- Prioritize weak and overdue words.
- Add mastery states.
- Add recall and usage question types.
- Add simple progress summaries.
- Add favorites/weak/due filters.

Dependencies: Reliable attempts, transactional updates, product definitions for mastery and daily boundaries.

Risks: A poorly explained scheduling algorithm can feel arbitrary.

Benefit: The application becomes a genuine learning tool.

Do not do yet: Highly configurable algorithms or complex gamification.

Phase 5 — Add advanced features / AI

Goals: Provide contextual coaching after the learning loop is proven.

Tasks:

- Add provider-agnostic AI service.
- Generate examples appropriate to a learner's level.
- Explain differences between meanings.
- Offer constrained practice conversations using selected words.
- Add opt-in storage, moderation, cost limits, and evaluation.
- Consider collections/import if usage data supports them.

Dependencies: Stable learning model, privacy decisions, usage telemetry, budget.

Risks: Cost, hallucinations, unsafe content, and low-value novelty.

Benefit: Personalized active practice.

Do not do yet: Autonomous tutoring or AI-generated mastery decisions without evaluation.

Phase 6 — Deployment and production maturity

Goals: Make releases repeatable and operations observable.

Tasks:

- CI builds/tests/security checks.
- Environment-based configuration.
- HTTPS and secure headers.
- Health checks and structured monitoring.
- Automated migrations with rollback procedure.
- Database backup/restore drills.
- Deployment runbook.
- Dependency update cadence.
- Load and resilience tests for dictionary and review endpoints.

Dependencies: Target hosting architecture and expected scale.

Risks: Environment differences and migration failure.

Benefit: Predictable releases and safer operations.

Do not do yet: Kubernetes or distributed services without demonstrated scale needs.

15. Prioritized Backlog

Order	Priorit y	Description	Why it matters	Size
1	Critical	Rotate JWT key and move secrets out of tracked settings	Prevents token forgery	Small
2	Critical	Replace SHA-256 password hashing with an adaptive hasher and migration path	Protects user credentials	Medium
3	Critical	Secure or remove <code>POST /api/words/add</code>	Prevents unauthorized data mutation	Small
4	Critical	Update <code>UserWord</code> learning counters/timestamps during quiz submission	Makes learning data truthful	Medium
5	High	Add authentication/ownership/quiz backend integration tests	Protects security boundaries	Large
6	High	Standardize success/error contracts and HTTP status mapping	Stabilizes every client interaction	Medium
7	High	Add global exception handling and request validation	Removes duplicated controller logic	Medium

8	High	Clean <code>UserWord</code> , <code>WordDefinition</code> , and stale DTO fields	Eliminates misleading schema/API behavior	Medium
9	High	Extract external dictionary client from <code>WordService</code>	Isolates an unreliable external dependency	Medium
10	High	Extract vocabulary commands/queries from <code>WordService</code>	Creates a home for notes, filters, and collections	Medium
11	High	Decompose <code>WordLookupComponent</code> and its template	Reduces UI regression and extension cost	Large
12	High	Replace static quiz-session storage	Enables restart and scale reliability	Medium
13	High	Handle concurrent dictionary cache misses atomically	Prevents intermittent unique-key failures	Medium
14	Medium	Add archive/delete vocabulary endpoint and UI	Completes collection management	Small
15	Medium	Add personal notes and user example CRUD	Immediate learning value from existing schema	Medium
16	Medium	Add auth interceptor and feature-specific Angular API services	Centralizes networking and typing	Medium
17	Medium	Restore server-side paging, filters, and debounced search	Supports collection growth	Medium
18	Medium	Add Playwright register/save/quiz smoke test	Protects the main product journey	Medium
19	Medium	Build a “Today” dashboard and persistent navigation	Creates a cohesive workflow	Medium
20	Medium	Add review-state schema and scheduling policy	Foundation for retention features	Large
21	Medium	Add daily due and weak-word queues	Delivers the core learning loop	Large
22	Medium	Add mastery and simple progress summaries	Makes improvement visible	Medium
23	Low	Fix accessibility, encoding, dead controls, and style-budget warning	Improves polish and usability	Medium
24	Low	Update README, test requests, and deployment documentation	Reduces developer confusion	Small

16. Top 5 Things I Would Do First

1. Rotate/externalize the JWT secret and replace password hashing.
2. Lock down the public dictionary-write endpoint.
3. Add backend integration tests around authentication, ownership, lookup, and quizzes.
4. Fix quiz submission so persisted attempts and per-word learning state agree.
5. Standardize API contracts, then split `WordService` and `WordLookupComponent` along dictionary/vocabulary boundaries.

These five actions address the largest security, correctness, and refactoring risks without changing the product's overall architecture.

17. Questions That Need Product/Developer Decisions

1. Is a saved vocabulary entry one word, one word-plus-part-of-speech, or one specific meaning?
2. What is the precise difference between saved, favorite, priority, mastered, and archived?
3. Should registration automatically sign the user in?
4. Should a user be allowed to edit a canonical definition, or only choose/add a personal interpretation?
5. Are personal examples distinct from provider examples, and should either be used in quizzes?
6. What counts as mastery: accuracy, streak, elapsed retention, or a combination?
7. What timezone defines a user's daily review and streak?
8. Should unanswered quiz questions count as incorrect?
9. Must an in-progress quiz survive refresh, restart, or use on another device?
10. Is the app intended for one English dictionary/language, or should multilingual support influence the model now?
11. Will there be real administrators? If so, what can they manage?
12. Is the existing hosted HTTP origin still active, and are there already production users requiring password-hash migration?
13. Should AI-generated content be stored, and can it influence review scheduling?
14. Is the near-term success metric saved words, daily reviews completed, retention, or returning learners?